

Row filters, column masks and dynamic views

Three ways to show different users different data from the same table, and when each is right.

All three work the same way: the query runs, and **the data is filtered or transformed at query time** based on who is asking.

The three mechanisms

MECHANISM	ATTACHED TO	BEST FOR
Dynamic view	A view over one or more tables	Curated or joined data for users with no access to the base tables
Column mask	One column on one table	Transforming a sensitive value in place
Row filter	One table	Restricting which rows a user sees
ABAC policy	A catalog or schema	Consistent rules across many tables. Module 30.

ON THE EXAM

Read the middle column. The scope each one attaches to is what really separates them, and what a scenario is usually pointing at.

The identity functions

- ◆ **is_account_group_member('group')** returns true if the querying user belongs to that account-level group.
- ◆ **current_user()** and **session_user()** return the identity running the query.
- ◆ **These are what make any of this dynamic.** Without them you have an ordinary view or a constant expression.
- ◆ **Account groups, not workspace-local groups**, which is module 28's Unity Catalog distinction again.

ON THE EXAM

The objective says 'based on user groups'. **is_account_group_member** is the function that phrase is pointing at.

Dynamic views

- ◆ **An ordinary view with an identity function inside it**, typically in a CASE expression or a WHERE clause.
- ◆ **Users are granted SELECT on the view**, and need no access to the base tables at all.
- ◆ **It can span several tables**, joining and reshaping as well as restricting.
- ◆ **They prevent users from modifying the underlying tables**, because the user never has access to them.

ON THE EXAM

A CASE returning either the real email or just its domain, depending on group membership, is the canonical example.

Dynamic views or filters and masks?

DYNAMIC VIEWS

A curated layer

- Can span and join multiple tables
- Fully support query optimisation and predicate pushdown
- Often better query performance
- Users need no access to the base tables

ROW FILTERS AND COLUMN MASKS

Applied to the table itself

- Protect the table wherever it is queried from
- Users cannot see base values before filtering
- Visible as policy metadata for auditing
- Scoped to one table, or one column

THE TIEBREAKER

The deciding question: should the protection follow the table, or should you present a different object entirely?

Two drawbacks of dynamic views

- ◆ **Limited auditing.** They lack semantic metadata such as tags or policy definitions in system tables.
- ◆ **So they are harder to audit at scale.** The rule lives in view SQL rather than in a policy you can query.
- ◆ **Vulnerability to probing.** They lack a SecureView barrier.
- ◆ **A user can craft a predicate with side effects** to infer information about the rows they cannot see.

ON THE EXAM

Both drawbacks are about governance rather than function. The view works; it is harder to prove what it does, and harder to defend.

A function that takes the column's value and returns **something of the same type** — the real value for some users, a redacted one for others.

Column mask syntax

STEP	STATEMENT
Create the function	<code>CREATE FUNCTION ssn_mask(ssn STRING) RETURN CASE WHEN is_account_group_member('HR') THEN ssn ELSE '***-**-****' END</code>
Apply to a column	<code>ALTER TABLE users ALTER COLUMN ssn SET MASK ssn_mask</code>
Or apply at creation	<code>CREATE TABLE users (name STRING, ssn STRING MASK ssn_mask)</code>
Remove	<code>ALTER TABLE users ALTER COLUMN ssn DROP MASK</code>

ON THE EXAM

A mask can take extra columns as parameters using `USING COLUMNS`, so the decision can depend on other values in the row.

A function that returns a **boolean**. Rows where it evaluates to true are visible; the rest are not returned at all.

Row filter syntax

STEP	STATEMENT
Create the function	CREATE FUNCTION region_filter(region STRING) RETURN is_account_group_member('global') OR is_account_group_member(region)
Apply to a table	ALTER TABLE sales SET ROW FILTER region_filter ON (region)
Remove	ALTER TABLE sales DROP ROW FILTER

ON THE EXAM

Note ON (region): you name which columns are passed to the function. That is how the filter sees the row's own values.

Limitations worth knowing

- ◆ **Path-based access to files in tables with policies is not supported.**
- ◆ **MERGE is not supported** where the policy contains nesting, aggregations, windows, limits, or non-deterministic functions.
- ◆ **Below Databricks Runtime 17.2**, DELETE, UPDATE and MERGE are unsupported on partitioned tables with a policy on the partition column.
- ◆ **Circular dependencies are not supported**, where a policy refers back to itself through another object.

ON THE EXAM

The path-based restriction is the most sweeping. A policy on a table cannot protect a file somebody reads directly.

The referencing restriction

- ◆ Row filters and column masks cannot reference tables that themselves have active row filters or column masks.
- ◆ So a lookup table used by a policy must be **unprotected**, or the policy will not work.
- ◆ This bites when you build a mapping table of users to regions and then protect that table too.
- ◆ In ABAC configurations there is a **workaround**: exclude the policy function owner from the referenced table's policy.

ON THE EXAM

A natural design — a policy that looks up entitlements in a table, and that table protected as well — is exactly what this forbids.

Where ABAC fits

- ◆ Row filters and column masks are per table and per column, managed by the table owner.
- ◆ That does not scale to consistent rules across hundreds of tables.
- ◆ ABAC policies attach at catalog or schema level and apply automatically based on governed tags.
- ◆ If both apply the same function, it works. Different functions, and access is blocked with an error — even if they would produce identical output.

ON THE EXAM

Module 30 covers ABAC properly. Know now that it exists, what it solves, and that mixing it with table-level policies has rules.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Hide a value but keep the row	A column mask
Hide the row entirely	A row filter
Users must not have access to the base tables	A dynamic view
Data must be joined from several tables first	A dynamic view
The rule must apply wherever the table is queried	A row filter or column mask, not a view
Consistent rules across many tables	ABAC policies. Module 30.
A policy stopped working after protecting a lookup table	Policies cannot reference protected tables

ON THE EXAM

Seven lines. The first two are the fundamental split, and the third and fifth are the pair that separates views from table-level policies.

A customers table holds a `national_id` column. Everyone in the company needs to query the table, and the row count must be correct for everyone. Only the compliance team may see the actual identifier values.

An engineer proposes creating a second table with the column removed, for everyone else to use.

What is the better approach?

- ☐ A The second table is correct; it is the only way to hide a column
- ☐ B A column mask on `national_id`, returning the real value for the compliance group and a redacted value for everyone else
- ☐ C A row filter on the customers table
- ☐ D Revoke `SELECT` on the table from everyone except compliance

◆ PAUSE AND DECIDE

What is the better approach?

- A

A second table without the column

Works on day one and creates a copy that must be kept in sync forever. Two tables mean two sets of grants and eventual drift.
- C

A row filter

Row filters remove rows. The requirement is that row counts stay correct, so removing rows is the wrong axis entirely.
- B

A column mask on national_id

Correct. One table, correct row counts for everyone, and the value transformed at query time based on group membership.
- D

Revoke SELECT from everyone else

Denies the whole table to people who legitimately need the other columns. Too blunt by an entire table.

THE SIGNAL

Two requirements pin it: everyone queries the table, and row counts must be right. That rules out filtering and rules out a separate copy.

A team must share a curated slice of data with an external-facing analyst group. The slice joins three tables, renames columns, and shows only aggregated figures. The analysts must not be able to reach any of the three base tables.

A colleague suggests applying row filters and column masks to all three tables instead.

Which approach fits?

- ☐ A Row filters and column masks on all three base tables, then grant SELECT on them
- ☐ B A dynamic view over the three tables, with SELECT granted on the view only
- ☐ C Copy the joined result into a new table on a schedule
- ☐ D Grant the analysts BROWSE on the three tables

◆ PAUSE AND DECIDE

Which approach fits?

- A

 Filters and masks on the base tables
Requires granting SELECT on all three, which the requirement forbids. It also cannot join or aggregate.
- B

 A dynamic view, SELECT on the view only
Correct. Views span multiple tables, reshape and aggregate, and users need no access to the base tables at all.
- C

 A scheduled copy
Works, and creates a second copy to maintain and secure, plus staleness between refreshes. A view is always current.
- D

 Grant BROWSE
BROWSE shows metadata without data. The analysts need the data, in a specific shape.

THE SIGNAL

'Must not reach the base tables' plus 'joins three tables' is the dynamic view signature. Neither is a filter-and-mask job.

A row filter on a sales table calls a function that looks up the querying user's permitted regions in a small `user_regions` mapping table. It has worked correctly for months.

A security review notes that `user_regions` itself lists who can see what, so a column mask is applied to it. The sales table filter immediately stops working.

Why?

- ☐ A The mask on `user_regions` changed the data type the filter expects
- ☐ B Row filters and column masks cannot reference tables that themselves have active row filters or column masks
- ☐ C Applying a mask invalidates all functions in the same schema
- ☐ D The filter needs to be dropped and re-applied after any change to a referenced table

◆ PAUSE AND DECIDE

Why?

- A

 The mask changed the data type
A mask must return the same type as the column it masks, so the type is unchanged by construction.
- B

 Policies cannot reference protected tables
Correct. This is a documented restriction, and the security review's change created exactly the situation it forbids.
- C

 Masks invalidate functions in the schema
Invented behaviour. A mask affects the column it is applied to, not unrelated functions.
- D

 Drop and re-apply the filter
Would not help. The restriction is structural, so re-applying the filter recreates the same forbidden reference.

THE SIGNAL

The security review was reasonable and produced a broken configuration. Protecting the lookup table is exactly what the restriction forbids.

A bank protects personal data across 340 tables using dynamic views, one per table, each embedding is_account_group_member checks. An auditor asks for a list of every column containing personal data and the rule applied to each.

The team cannot produce it without reading 340 view definitions by hand.

What does this illustrate?

- ☐ A Dynamic views lack semantic metadata such as tags or policy definitions in system tables, making them hard to audit at scale
- ☐ B Dynamic views do not actually restrict access, only appearance
- ☐ C The views should have been created in a single schema
- ☐ D Auditors cannot query system tables in Unity Catalog

◆ PAUSE AND DECIDE

What does this illustrate?

- A

 Limited auditing — no policy metadata in system tables
Correct. This is one of the two documented drawbacks of dynamic views. The rule lives in view SQL, which is not queryable as policy.
- B

 They do not really restrict access
They do restrict access properly. The problem is proving what they restrict, not whether they work.
- C

 Put them in one schema
Organises 340 views without making their logic queryable. The auditor still has to read every definition.
- D

 Auditors cannot query system tables
They can. The issue is that dynamic view logic does not appear there as policy metadata to be queried.

THE SIGNAL

The views work. What fails is governance at scale — which is precisely the gap ABAC policies and governed tags exist to close.

A mask returns the column's type and changes **what you see**. A filter returns a boolean and changes **which rows you see**. A view changes **which object you query**.